

Global placement with RePIAce

What it does Chooses approximate positions for the chip's logic cells so that connected cells are close, while cells are spread across the available space.

Remember this Connected cells pull together. Crowded regions push cells apart. Nesterov's method repeatedly updates their positions.

Where this fits in OpenROAD

After floorplanning, **global placement** distributes cells. Detailed placement then puts them on legal sites and removes overlaps. Clock tree synthesis and routing follow. Timing analysis is used repeatedly throughout the flow.

The algorithm in everyday language

Imagine arranging students in a classroom. Students doing a project together want nearby seats, but everybody cannot occupy the front row.

- 1 Start with cell positions.** Inputs include the netlist (which pins connect), cell sizes, fixed objects and usable chip area.
- 2 Estimate connection length.** Long connections receive a penalty. Moving connected cells closer usually reduces that penalty.
- 3 Measure crowding.** Divide the chip into small bins. Overfull bins create a spreading force, using an electrostatic model. These are mathematical charges, not real electrical charges.
- 4 Move cells with Nesterov updates.** Use the previous movement to form a look-ahead position, then use the slope of the cost there to choose the next move. This is accelerated gradient optimization.
- 5 Repeat and hand off.** Balance wire length against density until stopping criteria are reached. Output: approximate cell locations for legalization.

The two competing effects



Why it matters Poor placement creates long wires and congestion. Later timing repair and routing then have a harder problem to solve.

Global placement with RePIAce

One small numerical example

A and B are connected. A is at (0,0) and B at (10,0). Their simple Manhattan distance is 10 units. Moving B to (4,0) reduces it to 4 units.

But if many other cells already occupy that region, moving B there increases crowding. The placer must consider **both** effects, rather than just making every wire shorter.

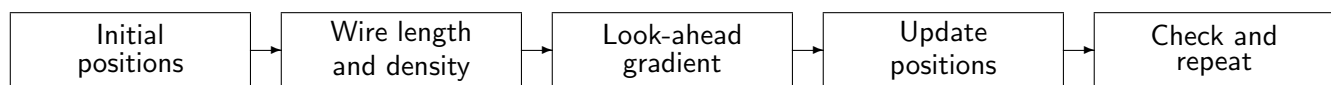
A simplified objective

$$\text{cost} = \text{smooth wire-length estimate} + \lambda \text{ density penalty}$$

Here λ controls how strongly crowding matters. OpenROAD uses a detailed numerical model; this expression explains the central idea.

Why Nesterov is useful

An ordinary gradient step looks for a downhill move from the current position. A Nesterov step uses momentum from previous movement and evaluates the direction at a look-ahead position. This can reach a useful placement with fewer iterations. It does not guarantee the best possible chip layout.



How timing and routing influence placement

Timing driven mode Gives more weight to connections with poor slack, encouraging the placer to shorten important connections. Slack means the time margin before a timing violation.

Routability driven mode Estimates routing congestion and can inflate the effective area of cells in crowded routing regions, encouraging more space there.

Tradeoff Spreading cells helps density and routing, but may lengthen wires. Pulling everything together can create overlap and congestion.

A command to recognize

```
global_placement -timing_driven -routability_driven
```

This is one stage command, not a complete script. The design, libraries, floorplan, timing constraints and wire RC model must already be set up.

What to say to your professors

“OpenROAD uses RePIAce for global placement. It treats wire length as a pulling effect and cell density as a spreading effect. Nesterov’s accelerated gradient method updates the cell positions. Detailed placement then makes those positions legal.”

Sources and code OpenROAD Global Placement documentation; Lu et al., ePlace, 2015. Implementation: `src/gpl/src/nesterovPlace.cpp` and `nesterovBase.cpp`. Examples in these handouts are illustrative, not measured OpenROAD results.

Static timing analysis with OpenSTA

What it does Calculates whether signals arrive early enough for setup checks and late enough for hold checks. It identifies the paths that need attention.

Remember this OpenSTA measures timing. It does not itself resize gates or build a clock tree. Its results guide those optimization engines.

The three times to understand

Arrival time When data reaches a pin, according to the delay model.

Required time The latest allowed arrival for setup, or the earliest allowed arrival for hold. Clock timing and constraints determine these limits.

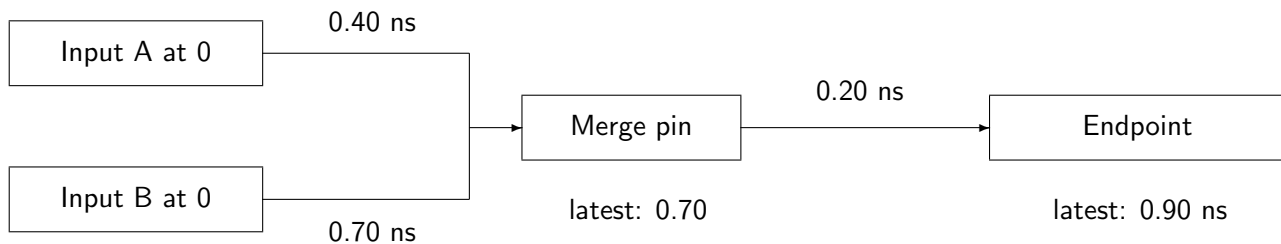
Slack The margin between arrival and the limit. Negative slack means that the timing check fails.

The algorithm is graph propagation

The timing graph has pins as nodes and cell or wire delays as edges. The tool propagates values through this graph instead of simulating every possible input waveform.

- 1 Read the timing model.** Use the netlist, Liberty cell delays, SDC timing constraints and estimated or extracted wire resistance and capacitance.
- 2 Propagate arrival times forward.** Add delay along each edge. At a merge, keep the latest relevant arrival for setup and the earliest relevant arrival for hold.
- 3 Propagate required times backward.** Start from endpoint limits and work backward through delays. This finds how much time is available at upstream pins.
- 4 Calculate slack and report.** Report critical paths, WNS and TNS. After a circuit change, incremental analysis updates affected timing values.

A tiny timing graph



For this simplified setup example, the merge takes $\max(0.40, 0.70)$. The final arrival is $0.70 + 0.20 = 0.90$ ns. Real analysis also distinguishes clocks, transitions and analysis conditions.

Static timing analysis with OpenSTA

Setup and hold use different slack signs

Setup: data must not be too late.

$$\text{setup slack} = \text{required time} - \text{latest arrival time}$$

If the latest allowed arrival is 0.80 ns and data arrives at 0.90 ns, slack is $0.80 - 0.90 = -0.10$ ns. The data is 0.10 ns late.

Hold: new data must not arrive too early.

$$\text{hold slack} = \text{earliest arrival time} - \text{required time}$$

If new data must not arrive before 0.18 ns but arrives at 0.12 ns, slack is $0.12 - 0.18 = -0.06$ ns. We need more minimum data-path delay.

WNS tells us the worst violation

WNS means worst negative slack. Consider four endpoint setup slacks:

$$A : -0.20 \quad B : -0.10 \quad C : +0.05 \quad D : -0.05 \quad \text{ns}$$

The WNS is -0.20 ns. Endpoint A has the largest timing failure.

TNS tells us the accumulated violation

TNS means total negative slack. Sum only the negative endpoint slacks:

$$\text{TNS} = -0.20 - 0.10 - 0.05 = -0.35 \text{ ns}$$

For the selected analysis, use each endpoint's worst relevant slack. Do not add every enumerated path to the same endpoint, and do not let positive slack cancel negative slack. Setup and hold are reported separately.

What zero means

OpenSTA's `report_wns` clamps positive worst slack to zero. If all checked endpoints pass, WNS and TNS are both zero. `report_worst_slack` can still show a positive margin. Correct constraints and all required operating corners must also be checked; zero in one report alone is not complete timing closure.

Commands and a short spoken explanation

```
report_checks -path_delay max
```

```
report_wns -max      report_tns -max
```

Use `-min` for hold WNS/TNS; a detailed hold report uses `-path_delay min`.

"OpenSTA propagates arrival and required times through a timing graph. Slack tells us whether a check passes. WNS is the worst negative margin; TNS adds the negative margins across endpoints."

Sources and code Official OpenSTA repository and documentation. Definitions checked against `src/sta/search/Search.tcl` and `Search.cc` in the local OpenROAD checkout. Values above are teaching examples with already determined required times.

Clock tree synthesis with TritonCTS

What it does Builds a buffered network that delivers a clock signal to many flip-flops with controlled delay, skew and signal transition time.

Remember this Divide clock sinks into groups, build balanced branches, and insert suitable clock buffers.

Why one clock wire is not enough

A clock can drive thousands of flip-flops. One driver connected to every clock pin sees a large electrical load. Long wires also add resistance and capacitance. Clock edges become slow, and distant sinks may receive them at different times.

Think of a school announcement system: local amplifiers distribute the same bell signal to groups of classrooms.

Four words you should know

Sink A receiving clock pin, such as a flip-flop clock input.

Buffer A cell that strengthens and redistributes the clock signal.

Clock latency The travel time from the clock reference to a sink.

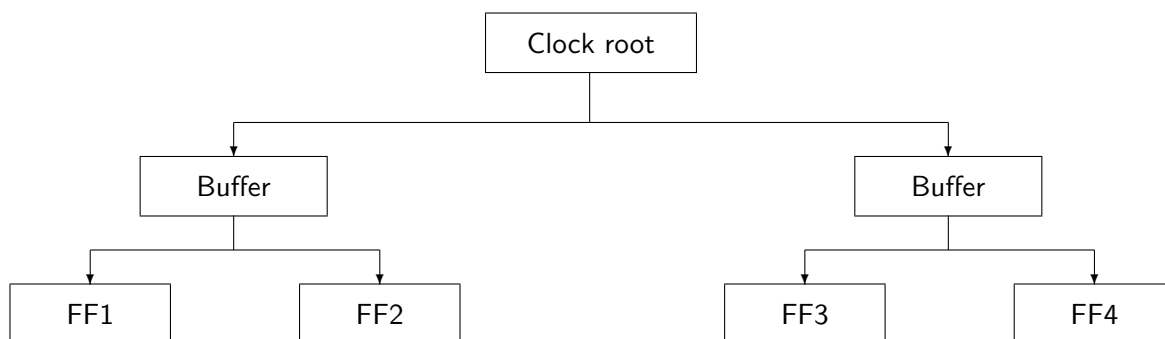
Clock skew The difference in clock arrival times between sinks.

The main algorithmic ideas

OpenROAD uses **TritonCTS 2.0**. It characterizes candidate buffer and wire segments using the loaded technology data, then uses this information while building the tree.

Clustering groups sinks spatially. Capacitated K-means, called **CKMeans** in the code, supports controlled partition sizes while building branches. An **H-tree based hierarchy** distributes the clock toward sink groups. Optional sink pre-clustering adds local buffered groups before that tree is built.

A simplified buffered clock tree

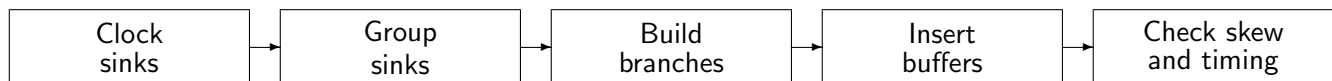


FF means flip-flop. This diagram shows connectivity, not an exact physical H shape. Real trees adapt to sink positions, loads, macros and constraints.

Clock tree synthesis with TritonCTS

The steps from input to output

- 1 **Gather sinks.** Read clock definitions, sink positions, buffers and wire RC data.
- 2 **Form branches.** Partition sinks and construct an H-tree based hierarchy.
- 3 **Choose buffers.** Use characterization to control capacitance and slew, meaning the time taken by a signal edge to transition.
- 4 **Balance and place.** Build buffered subnets and balance loads or latencies where supported and enabled.
- 5 **Recheck timing.** Legalize placement, update parasitics, propagate clock delays, and check setup and hold.



A small skew example

Before balancing, suppose the clock arrives at four sinks at:

0.10, 0.18, 0.25, 0.30 ns

The overall arrival spread is $0.30 - 0.10 = 0.20$ ns.

After CTS, suppose arrivals are:

0.22, 0.23, 0.24, 0.23 ns

The spread is now $0.24 - 0.22 = 0.02$ ns. Some sinks became later, but delivery became much more balanced. These are illustrative numbers, not a promised CTS result.

How clocks affect WNS and TNS

For a register-to-register path, both the launching and capturing clock arrival times affect slack. A later capture clock can help setup but hurt hold. CTS therefore needs timing checks after the actual clock network is modeled.

Equal wire length alone does not guarantee equal delay.

Tradeoff More buffers can improve drive strength and balance, but add area, clock power and insertion delay. CTS aims for acceptable skew and electrical limits, not necessarily zero skew.

Commands and a short spoken explanation

```
clock_tree_synthesis    report_cts
```

The platform must supply valid clock buffers and RC settings. `report_cts` reports tree counts; timing reports are needed to assess clock arrival times and slack.

“TritonCTS groups clock sinks, builds an H-tree based branching network, and inserts buffers using technology delay information. The aim is a well-driven clock with controlled skew, followed by setup and hold analysis.”

Sources and code OpenROAD Clock Tree Synthesis documentation. Code: `src/cts/src/HTreeBuilder.cpp`, `Clustering.cpp`, `SinkClustering.cpp`, `TechChar.cpp`.

Timing repair with the Resizer

What it does Changes cells and buffering to reduce setup and hold violations, using OpenSTA to measure the effect of each round of repair.

Remember this Measure timing, find a bad path, change the circuit, and measure again. WNS and TNS are the scores that guide this loop.

How WNS and TNS guide repair

WNS identifies the worst negative slack. **TNS** adds negative endpoint slacks. The command `repair_timing` uses repair heuristics to improve these metrics. A heuristic is a practical search strategy, not a guarantee of the global optimum.

Setup repair means making late data faster

The Resizer targets the worst setup path and then works on other violating endpoints to reduce total negative slack. Depending on the library and settings, it can use these changes:

Gate sizing Replace a cell with a stronger equivalent to drive its load faster. A larger cell also adds area and input capacitance.

Buffering or removing buffers Split a large load or long wire into better-driven sections; remove an unnecessary buffer if its extra delay hurts.

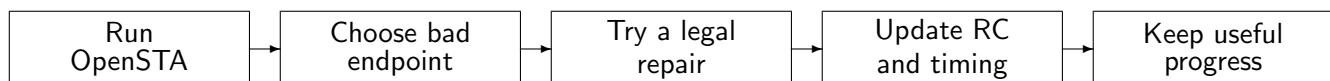
Cloning or load splitting Give a critical load a less heavily loaded driver by duplicating suitable logic or reorganizing buffering.

Pin or threshold-voltage swaps Use a faster equivalent input pin or an available faster cell variant. Only legal, function-preserving changes are allowed.

Hold repair means slowing early data

If data arrives too early, hold repair typically inserts delay buffers in the **data path**. The extra minimum delay can fix hold, but consumes setup margin. OpenROAD normally repairs setup first and guards setup while inserting hold buffers.

The repair loop



Repeat until timing targets are reached, progress stops, or area, buffer and iteration limits prevent further repair. The tool may undo changes that fail its improvement criteria.

Inputs A timed physical design, libraries, constraints and parasitics.

Outputs A modified netlist and cell placement needing legalization, routing updates and timing verification.

Timing repair with the Resizer

Why improving only WNS is not enough

Suppose three endpoint setup slacks are:

Before: $-0.20, -0.10, -0.05$ ns

WNS is -0.20 ns and TNS is -0.35 ns.

Repair the first endpoint so its slack becomes $+0.02$ ns:

After: $+0.02, -0.10, -0.05$ ns

WNS improves to -0.10 ns and TNS to -0.15 ns. **Two endpoints still fail.** Further repair is required. In a real circuit, one change can affect several endpoints at once.

A setup fix and a hold fix

Setup example A path has a 1.00 ns allowed arrival and arrives at 1.12 ns: slack = -0.12 ns. A suitable stronger driver reduces arrival to 0.96 ns: slack becomes $+0.04$ ns.

Hold example The earliest allowed arrival is 0.12 ns, but data arrives at 0.08 ns: slack = -0.04 ns. A delay buffer adds 0.06 ns: arrival becomes 0.14 ns and hold slack becomes $+0.02$ ns. Setup must be checked again.

Both are simplified teaching examples; real cell and wire delays depend on load, slew and operating conditions.

Commands to recognize

With CTS complete, propagated clocks and valid parasitics, the essential repair commands are:

```
repair_timing -setup -repair_tns 100
```

```
repair_timing -hold
```

`-repair_tns 100` targets all violating endpoints; `0` targets only the worst endpoint. The number is a percentage of endpoints to attempt, not a promised percentage improvement. Check `help repair_timing` in your installed version.

When can you claim timing closure

Re-legalize changed cells, update routes and parasitics, then rerun setup and hold analysis. Check all required timing corners, constraints and electrical limits. A clean pre-route estimate can fail after real wire delays are included. Area, power or congestion may also limit repair.

What to say to your professors

“OpenSTA measures WNS and TNS. The Resizer improves them by trying gate and buffer changes on critical paths. Setup repair speeds up late data; hold repair delays early data. We repeat analysis and repair with updated physical information until all required checks pass.”

Sources and code OpenROAD Gate Resizer documentation, Repair Timing. Code: `src/rsz/src/Optimizer.cc`, the setup policies in `src/rsz/src/policy/`, and `RepairHold.cc`. Command behavior checked against local source.

Global routing with FastRoute

What it does Plans which coarse chip regions and metal layers each signal connection should use while trying to avoid routing congestion.

Remember this Start with short routes. Make crowded corridors expensive. Reroute selected connections through less crowded corridors.

A road map for wires

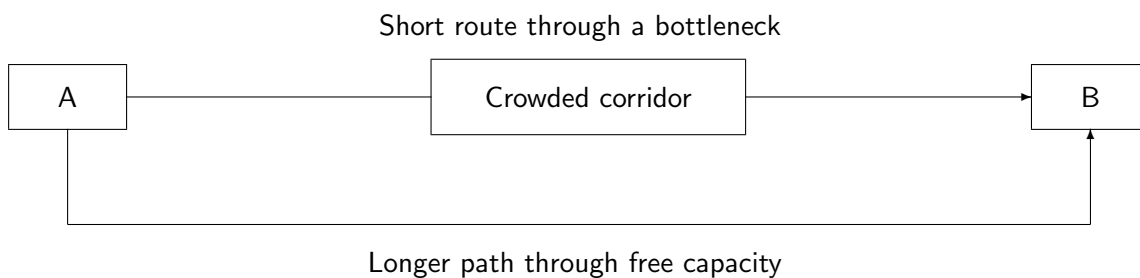
Think of routing as planning roads between houses. The shortest road is attractive, but not if every vehicle needs the same narrow bridge.

OpenROAD's **FastRoute based global router** divides the chip into a coarse grid. Connections cross grid edges with limited routing capacity. A **net** is a set of pins that must be electrically connected. A **via** connects wires on different metal layers.

The important algorithmic steps

- 1 Build the routing grid.** Read placed pins, allowed layers, obstacles and estimated capacity for wires between neighboring regions.
- 2 Create short connecting trees.** Steiner-tree construction can introduce extra branch points to connect a multi-pin net with less total wire. FastRoute uses fast pattern routing for initial connections.
- 3 Find overloaded corridors.** Compare routing demand with capacity. Demand above capacity is congestion overflow.
- 4 Rip up and reroute.** Remove selected route segments. A maze search explores grid neighbors using costs that discourage congestion and excessive wire or vias. Repeated congestion penalties encourage alternate paths.
- 5 Assign layers and produce guides.** Refine the routing plan and hand coarse guides to detailed routing. Detailed routing selects exact tracks and vias and checks physical design rules.

Why the shortest route may lose



This is a conceptual route map. The actual search uses a grid, many interacting nets and several routing layers.

Global routing with FastRoute

One corridor with too many wires

A grid edge has capacity for 5 wire tracks. The current routes need 8. Its overflow is:

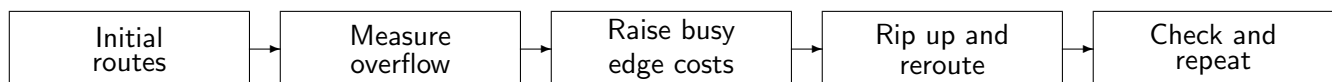
$$\max(0, \text{demand} - \text{capacity}) = \max(0, 8 - 5) = 3$$

If three connections can be moved to another corridor with spare capacity, demand falls to 5 and this edge's overflow becomes zero. The router must also ensure the detour does not simply create a new bottleneck elsewhere.

How maze routing chooses a path

Each grid move has a cost. Search accumulates the costs while exploring possible routes. A path with more grid steps can win if it avoids expensive congested edges.

A teaching example A short route has length cost 4 plus congestion penalty 10: total 14. A detour has length cost 7 and no congestion penalty: total 7. The search prefers the detour. These are invented scores, not FastRoute's exact cost formula.



How routing affects timing closure

Routing determines wire length, layer choices and vias. These change wire resistance and capacitance, which change signal delay. OpenSTA uses updated parasitics to recompute slack; the Resizer can then repair newly visible violations.

Tradeoff A detour may reduce congestion but worsen timing. Timing-critical nets can be prioritized. If the layout is too dense, routing alone may not solve the problem; placement or the floorplan may need revision.

Commands to recognize

```
global_route
```

```
estimate_parasitics -global_routing
```

These require an initialized physical design and valid routing and RC settings. The first produces a global route plan; the second estimates its parasitics for timing analysis. Exact metal geometry is created later by the detailed router, commonly called TritonRoute in OpenROAD.

What to say to your professors

"FastRoute plans connections on a coarse grid. It starts with short routes, detects overused routing corridors, and uses rip-up and maze rerouting to reduce congestion. The output guides detailed routing and provides better wire-delay estimates for timing repair."

The full loop Place cells, build the clock, route wires, analyze timing and repair. Timing analysis and repair repeat across these stages.

Sources and code OpenROAD Global Routing documentation; Pan et al., FastRoute, 2012. Code: `src/grt/src/fastroute/src/FastRoute.cpp`, `maze.cpp`, `maze3D.cpp`. Source review: 7 September 2026; local OpenROAD revision 63fe72c577ef.